

Serial Protocol Decode — DMM6500

A UART decoder that runs **on** a Keithley DMM6500 bench multimeter. It digitizes the line with the instrument’s own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.03 — beta** · MIT licence (see LICENSE) · user manual

Beta status. Everything the manual claims was measured on the bench, and the release gate below is what measures it — it must pass with zero failures before a build goes out. One rough edge worth naming here: **flow control** is verified electrically — 4.92 V, ~10 μ s, one credit pulse per armed capture — but has never been closed as a loop against a device that actually waits for it, so the “unlimited with flow control” claim is sound by construction rather than demonstrated. The pulse width is **not settable** on this firmware — `trigger.extout.pulsewidth` is nil on 1.7.17a — and the default measures **9.4–9.8 μ s**, so a device waiting for a credit needs an edge-triggered input rather than a polling loop.

Endurance, measured. An 8-hour soak of this build on one power cycle ran **98 laps — 4,214 capture-and-decode cycles and 9 one-press recordings — with zero wrong answers**. Every recording ended because its buffer filled, at 8 192 bytes, with no instrument events logged and no error left behind. The number worth more than the failure count is the **lap time**: a mean of 300.9 s over the first ten laps and 292.9 s over the last ten, drifting **–2.7 %** across the full eight hours, so nothing accumulated. A leaked display object, a buffer handle that was never returned or a fragmenting allocator would all have shown up as laps getting *slower*, and none did — which matters because a leak here would not have failed a lap, it would have wedged the instrument around hour six. The soak also settles a specific defect: the 2× baud misfit that made 9600 read as 19200 affected **15.2 %** of laps in v1.01 and recurred **zero** times in 98.

Eleven of those 98 laps are printed as failures by `tools/soak.py`, and none of them is one. All eleven are the same measurement artifact in the *harness*: the long-payload gate scores a capture on its longest unflagged run, which is an order statistic, so **one honestly-flagged byte landing more than ten bytes from either edge fails the point** however few there are — at 239 bytes only 22 of 239 single-flag positions can pass. The bytes the decoder stood behind were byte-exact every time — as they were in all 143 long-payload points ever recorded here. The gate has since been **replaced** by one that is *stricter*, not laxer: it validates **every** clean run against the payload at one agreed alignment instead of only the longest, and bounds the flag **count**, which does not move when a flag changes position. It catches two things the old rule passed — a corrupted byte inside a short run, and a decode that lost and regained sync mid-capture. `tools/rejudge_soak.py` re-judges the soak records either way.

What that soak does **not** cover: it is depth rather than breadth — 43 stimulus points repeated 98 times, no front-panel interaction (that is hw-panel’s job), and one power cycle rather than many.

Nothing stops a long job early — there is no working stop control. A touch press cannot be delivered while a script runs, so the app cannot show a Cancel button, and the front-panel TRIGGER key does not fill the gap: it does **not** deliver `trigger.EVENT_DISPLAY` while a PANEL-initiated run is executing, measured on hardware 2026-08-18. `tools/bench_cancelkey.py`’s

contrary result — a press during a 10 s Lua spin still seen by the poll afterwards, 1.06 ms per poll, zero event-log entries — was a HOST-initiated run, which is a different execution context, so it does not license the panel claim. So a recording runs for its stated time and cannot be interrupted; decide the size before pressing. The user manual says so under **The TRIGGER key**. The same key does still *gate* an armed capture when `Options ▶ Trigger = Trigger` key, which took its own purpose-built test (`tools/bench_trigkey.py --quiet-line`) because timing-based tests could not tell a prompt press from a free-running capture.

How much it captures, and how many presses. A screenful is ~240 bytes; a recording holds **8 192 or 32 768 bytes** to a file, and **one press** records it, decodes all of it and files it — no stepping, where a full buffer once cost twelve presses. Beyond one window, flow control makes the window a chunk size rather than a total: one press then credits the device, records, decodes and credits again until it stops sending, so an unlimited amount can be captured losslessly **with no interaction per window** by a device built to wait for the credit — bounded at 32 windows or 20 minutes per press, then Mode and Capture resume the same conversation in the next file — the sender is still waiting for its credit, so nothing is lost in the gap. There is deliberately no uncapped mode: continuous decode-to-file is drain-bound at 2400 baud, too slow to be worth a mode. The arithmetic is in docs/REFERENCE.md.

Ships as `Serial_Decompile.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the instrument firmware, which is why the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

Before releasing it to anyone

`python3 tools/release_sweep.py`

That is the gate. It runs every check in one go and writes an auditable record to `out/release/<timestamp>/` — a `REPORT.md`, a `summary.json`, the full output of every stage, every front-panel screenshot taken, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-zero if any gate fails.

It needs a freshly power-cycled DMM6500. `sdec.start()` builds the display objects and the firmware does not fully return the pool, so the app refuses a second build in one power cycle. The sweep checks this up front and refuses with the remedy rather than failing forty minutes in. It also needs the SDG2122X generator with the stimulus waveforms loaded (`bench_matrix.py --upload` puts them there) and only one client may hold the DMM's control socket at a time.

For a code change, the offline half runs in under ten seconds and needs no instruments:

`python3 tools/release_sweep.py --offline`

Stage	Checks
lint parse	Lua 5.0.2 incompatibilities; luac -p on every module
unit	938 offline tests — decoder, UI, state machine, file paths, every visible ASCII glyph
unit-cancel	the TRIGGER-key cancel latch, one-press recordings, both window sizes, the flow-control loop
stress	hostile signals: never silently wrong , never raises
unit-loremgate	the harness's own long-payload verdict — every clean run validated, flag count bounded
tolerance	recomputes the envelope table printed in the manual
package archive	rebuilds the .tspa, then builds both screens <i>from the archive</i> against a mock front end
manual	rebuilds every shipped PDF: docs/MANUAL.pdf, docs/REFERENCE.pdf, README.pdf
hw-matrix	on the bench, through the app's own Capture button: six frame formats, the standard rate ladder, a 1 kB non-repeating payload, three logic swings, twelve DC offsets
hw-odd-rates	nineteen non-standard baud rates — 900, 1500, 3600, 8123, 29127, 104857 ...
hw-panel	every button in every state, with the panel grabbed before and after each press and differenced by region — including a one-press recording and a cancel delivered mid-handler by a trigger timer, which is the only way a harness can exercise the cancel path at all
hw-break	degenerate signals and contradictory settings — no signal, DC only, all-0x00/0xFF/0x55, a break, 60 mV of swing, 19 Vpp, rates past the ceiling, and six wrong forced settings. A refusal with a reason passes; confident garbage does not

hw-panel is the one worth understanding: it checks six things per press — that the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, *and that the panel actually shows it*. The last is a separate failure from the one before it, because the UI caches its writes: an app can be right while the operator reads something stale.

Layout

tsp/	the app. serial_core acquisition, uart_decode framing, chunk_decode resumable decode, serial_ui panel, serial_app orchestration
tools/	harnesses. release_sweep.py is the entry point; the rest are the individual authorities it calls
docs/	the manual, instrument references, panel mockups
notes/	bring-up log, findings, handoffs

tsp/midi_decode.tsp and tsp/lin_decode.tsp are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in tools/package_tspa.py; the app discovers what it has at runtime.

Bench

Three instruments over LAN: the DMM6500 under test, an SDG2122X generating the stimulus, an SDS1204X-E for verifying the stimulus is what it claims to be. tools/instruments.py holds the addresses and the hazards worth knowing — repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.